

ANÁLISIS DE COMPONENTES PRINCIPALES (ACP) - UNA GUÍA PRÁCTICA CON R PARA LAS CIENCIAS QUÍMICAS

Juan Pablo Carreón-Hidalgo

Facultad de Ingeniería Química, Benemérita Universidad Autónoma de Puebla, 72570 Puebla, México

*e-mail: jpch_26@outlook.com

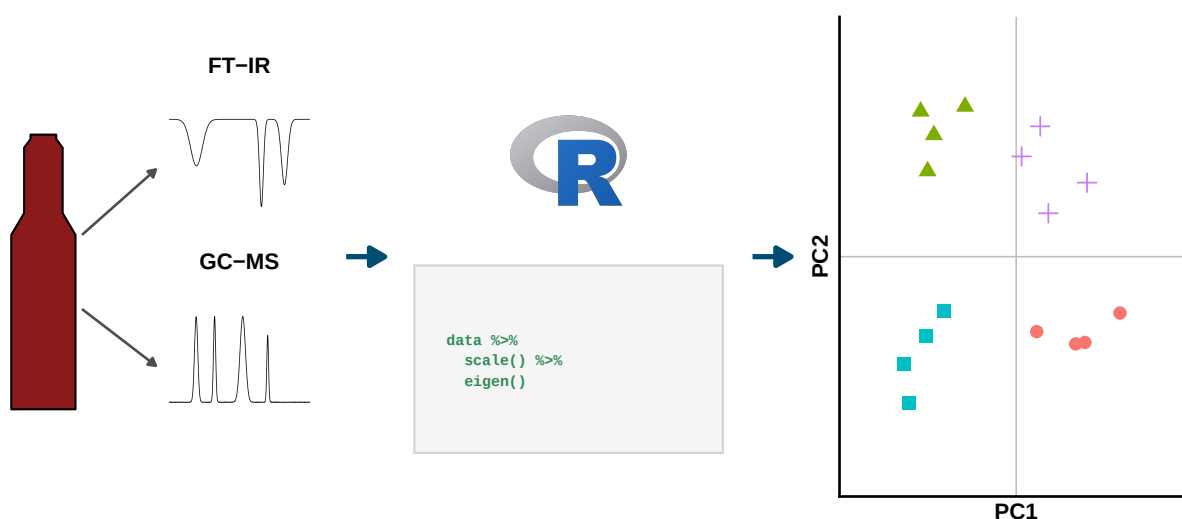


Figure 1: Graphical Abstract

Esta guía educativa ofrece un tutorial paso a paso para aplicar el análisis de componentes principales (ACP) en R. Mediante un flujo de trabajo práctico, se analizan con éxito conjuntos de datos analíticos (FT-IR y GC-MS) de muestras de vino internacionales con el fin de reducir la dimensionalidad y realizar una discriminación geográfica.

RESUMEN

El Análisis de Componentes Principales (ACP) es una herramienta fundamental dentro del análisis multivariable, el cual tiene aplicaciones directas en las ciencias químicas para la exploración y simplificación de datos masivos o de alta dimensionalidad. Este artículo de difusión presenta una guía práctica y metodológica para comprender los fundamentos subyacentes al ACP, implementando un flujo de trabajo reproducible en el lenguaje de programación R. A través de un enfoque secuencial, se desarrolla primero un escenario didáctico con datos en dos dimensiones para ilustrar los procesos críticos de centrado, escalado y descomposición matricial. Posteriormente, se aborda un caso de estudio real enfocado en la caracterización química y origen geográfico de muestras de vino evaluadas mediante cromatografía de gases acoplada a masas (GC-MS) y espectroscopia infrarroja con transformada de Fourier (FT-IR). Los resultados demuestran la utilidad del

ACP para reducir dimensiones, identificar correlaciones entre variables y visualizar agrupamientos analíticos de manera intuitiva y global.

Palabras clave: Análisis de componentes principales (ACP); Programación en R; Análisis multivariante; Ciencias químicas; Reducción de dimensionalidad

INTRODUCCIÓN

El análisis de componentes principales (*ACP*) es un método estadístico que permite combinar la información de distintas variables observadas o medidas en los mismos sujetos o unidades experimentales para obtener un menor número de variables denominadas componentes principales (*CP*). Los *CP* explican de manera óptima la mayor parte de la variación (o varianza) observada en los datos originales y estos tienen propiedades geométricas que permiten una interpretación intuitiva y estructurada de las principales características en un conjunto de datos multivariable complejo. (Greenacre et al., 2022) Llevado a un contexto práctico, este método permite reducir el número de dimensiones de un conjunto de datos para visualizar posibles relaciones o correlaciones de manera global. Por ejemplo, si se realizan mediciones de muchas variables respuesta en una misma unidad experimental y además se cuenta con varias unidades claramente diferenciadas por distintas categorías o por la aplicación de distintos tratamientos es muy posible que esto genere un enorme conjunto de datos. Visualizar estos implicaría generar un número considerable de gráficas en dos o tres dimensiones mostrando las relaciones entre las variables respuesta y los tratamientos aplicados, lo que sería difícil de interpretar en conjunto.

Es importante mencionar que el *ACP* solo permite obtener una interpretación descriptiva (o exploratoria) del conjunto de datos así como de las posibles correlaciones entre variables respuesta y las diferencias entre categorías o tratamientos aplicados. No se trata de un método inferencial como el *ANOVA* o las pruebas clásicas de comparación entre medias. Estrictamente, el *ACP* se puede incluir dentro lo que se denomina análisis multivariable. Los métodos dentro de esta denominación permiten analizar simultáneamente medidas múltiples de cada individuo u objeto sometido a investigación. (Fernández-Chuairey et al., 2022)

Fundamento matemático breve del ACP

El *ACP* es un método de proyección lineal basado en la premisa de que la información más importante en un conjunto de datos reside en su variabilidad. Con esto en mente el objetivo principal de este método es encontrar un nuevo sistema de coordenadas donde los nuevos ejes, llamados *CP*, recojan la varianza de mayor a menor en el sistema original. (Wold et al., 1987)

Desde un punto de vista algebraico, cada CP_j se puede expresar como una combinación lineal de las variables originales $X_1, X_2, \dots, X_p$. La expresión general sería:

$$CP_j = c_{1j}X_1 + c_{2j}X_2 + \dots + c_{pj}X_p \quad (1)$$

donde los coeficientes c_{ij} se denominan cargas o *loadings*. Estos valores representan el peso o la importancia de cada variable original en el CP_j correspondiente. Además, se impone una restricción de normalización donde la suma de los cuadrados de las cargas debe ser igual a uno:

$$\sum_{i=1}^p c_{ij}^2 = 1 \quad (2)$$

En el caso del primer componente principal (CP_1), este también se define como la dirección en el espacio que captura la máxima variación posible de los datos proyectados (datos originales). El segundo componente (CP_2) debe capturar la mayor parte de la variación restante bajo la condición estricta de ser perpendicular (ortogonal) al primero, y así sucesivamente para cada componente. Al cumplirse dicha condición se asegura que no exista correlación entre los componentes.

La solución para encontrar los CP se basa en la descomposición de la matriz de varianza-covarianza (Σ) o la matriz de correlación (R) cuando se trata de variables estandarizadas. Matemáticamente, las direcciones de los CP corresponden a los vectores propios de dichas matrices y la varianza explicada por cada uno es igual a su valor propio (λ) asociado. Al ordenar los valores propios de mayor a menor ($\lambda_1 > \lambda_2 > \dots > \lambda_p$) se establece el orden de importancia de los componentes.

A continuación se ofrece un tutorial con R (R Core Team, 2026) cuyo objetivo es ofrecer las bases y código necesarios para comprender y aplicar el ACP . Primero se presenta un ejemplo sencillo en dos dimensiones que posteriormente permite generalizar los principios a un caso con datos con muchas dimensiones provenientes de muestras de vino de distintos países. (Bro & Smilde, 2014) Es posible, y muy recomendable, reproducir este tutorial utilizando la consola de R o en una sesión de $RStudio$. También es recomendable introducir variaciones en el código y observar los resultados o intentar aplicarlo a sus propios datos experimentales.

CASO SIMPLE CON DATOS EN DOS DIMENSIONES

Instalación y carga de paquetes

A lo largo de todo el tutorial se utilizarán paquetes de funciones que facilitan varias tareas. Para instalar dichos paquetes ejecutamos el siguiente código:

```
# Cargar los paquetes en su sesión de R
install.packages("ggplot2") # Gráficas
install.packages("ggpubr")  # Gráficas conjuntas
install.packages("dplyr")   # Manipulación de datos
install.packages("readr")   # Importación o descarga de datos
install.packages("purrr")   # Programación funcional para operaciones sobre columnas
install.packages("stringr") # Manipulación de cadenas de caracteres
```

Si ya se encuentran instalados se puede omitir lo anterior y pasar directamente al cargado de paquetes.

Para la primera parte del tutorial se utilizará `ggplot2` para realizar gráficas y `dplyr` para manipular datos de forma eficiente. (Wickham et al., 2019)

```
# Cargar los paquetes en su sesión de R
library(ggplot2)
library(ggpubr)
library(dplyr)
```

Datos simulados

Primero simulamos dos variables con 50 valores cada una. En el caso de la primera cada valor asignado es un número aleatorio generado por la función `rnorm()`, mientras que para la segunda variable establecemos una relación lineal con la primera con el objetivo de asegurar una correlación.

```
set.seed(1) # Para reproducibilidad

# Variable 1
var1 <- rnorm(50, 50, sd = 3)

# Variable 2
var2 <- 5*var1 + rnorm(50, sd = 10)
```

```
# Ambas variables en un marco de datos ("data frame")
datos_simulados <- data.frame(var1, var2)
```

Para visualizar los datos puede utilizar la función `View()` o de forma breve `head()`. El argumento en ambas funciones es el conjunto de datos que deseamos visualizar; por ejemplo, en este caso ejecutemos `View(datos_simulados)` para visualizar el marco de datos de forma completa en una ventana nueva. Cabe resaltar que lo anterior se recomienda para todos los datos utilizados y generados en este tutorial.

Al realizar una gráfica de dispersión de nuestros datos simulados es posible apreciar la relación que establecimos entre ambas variables.

```
# Gráfico de dispersión con las dos variables simuladas
ggplot(datos_simulados, aes(x = var1, y = var2)) +
  geom_point(color = "#EE5C42", size = 2) +
  xlab("Variable 1") +
  ylab("Variable 2") +
  theme_classic()
```

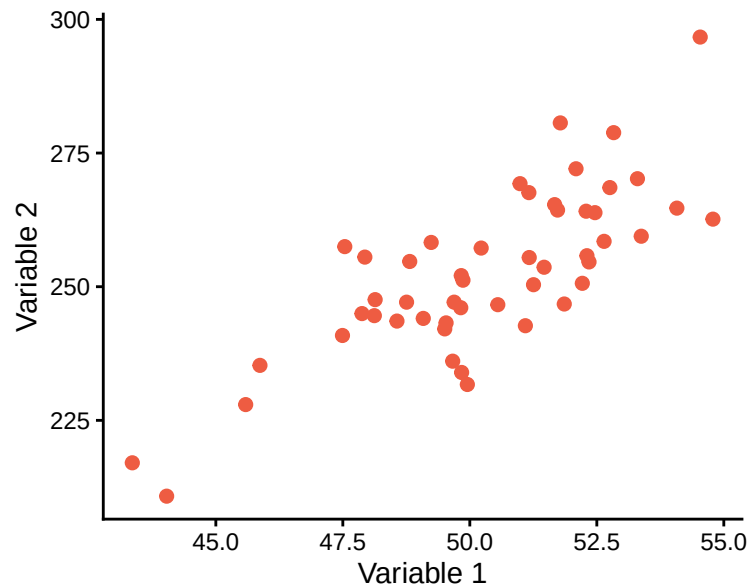


Figure 2: Gráfico de dispersión de las variables simuladas originales.

Centrado y escalado de cada variable

Cuando llevamos a cabo un *ACP* es común que las variables respuesta para cada muestra o individuo se encuentren en distintas unidades y rangos de magnitud. Con esto en mente cabe mencionar que el *ACP* es muy sensible a la escala de los datos; por ejemplo, si en nuestro conjunto de datos hay una variable con valores numéricamente grandes en comparación con el resto, esta “dominará” el análisis aunque no sea la más informativa. Para evitar lo anterior realizamos dos pasos críticos: el centrado y el escalado de los datos de cada variable. (Greenacre et al., 2022)

El centrado consiste en restar la media de cada variable a todos sus valores correspondientes:

```
# Restar la media de cada variable
datos_centrados <- datos_simulados %>%
  mutate(
    # Añadir dos columnas nuevas al marco de datos
    var1_c = var1 - mean(var1), # Variable 1 centrada
    var2_c = var2 - mean(var2) # Variable 2 centrada
  )
```

La operación anterior traslada el origen del sistema al “centro de gravedad” de la nube de datos, pero no modifica la posición relativa entre cada punto, por lo que la apariencia de nuestros datos en una gráfica de dispersión no cambia:

```
# Gráfico de dispersión para los datos centrados
ggplot(datos_centrados, aes(x = var1_c, y = var2_c)) +
  geom_point(color = "#EE5C42", size = 2) +
  geom_vline(xintercept = 0, linewidth = .5) +
  geom_hline(yintercept = 0, linewidth = .5) +
  xlab("Variable 1 centrada") +
  ylab("Variable 2 centrada") +
  theme_classic()
```

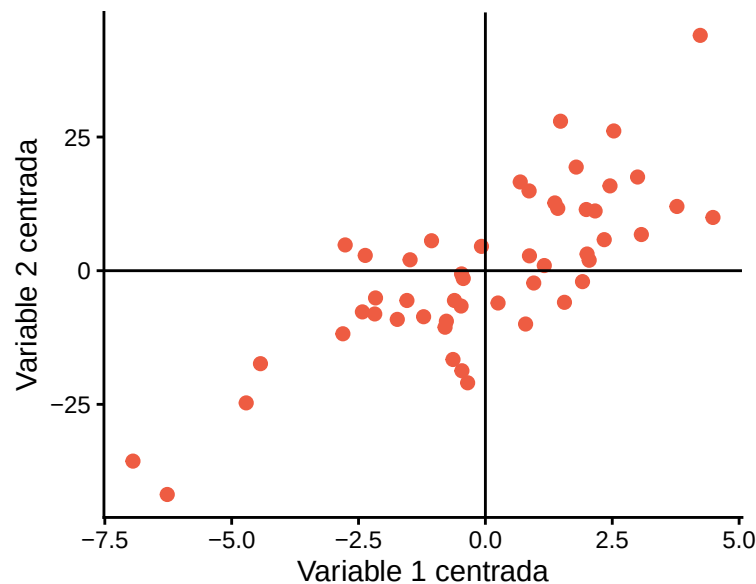


Figure 3: Gráfico de dispersión de los datos simulados tras el proceso de centrado respecto a la media.

Con el objetivo de dar la misma importancia a cada variable realizamos un escalado o *autoscaling*, lo que se traduce en dividir cada variable centrada entre la desviación estándar de los datos originales:

```
# Restar la media de cada variable
datos_estandarizados <- datos_centrados %>%
  mutate(
    # Añadir dos columnas nuevas al marco de datos
    var1_e = var1_c/sd(var1), # Variable 1 escalada
    var2_e = var2_c/sd(var2) # Variable 2 escalada
  )
```

Tras el centrado y escalado, las variables estarán estandarizadas y tendrán una varianza igual a 1 (resultado

del escalado) y una media de 0 (resultado del centrado). En términos generales lo que hacemos con ambas operaciones es lo que comúnmente denominamos calcular puntuaciones Z:

$$z = \frac{x - \mu}{\sigma} \quad (3)$$

Cálculo de la matriz de covarianza

Una vez que centramos y escalamos los datos procedemos a calcular la matriz de covarianza para las dos variables. Para esto podemos utilizar de forma directa la función `cov()`.

```
# Seleccionar solo las variables centradas y escaladas
datos_estandarizados<- datos_estandarizados %>%
  select(var1_e, var2_e) %>%
  as.matrix()

# Calcular la matriz de covarianza
cov_m <- cov(datos_estandarizados)
```

La diagonal de la matriz de covarianza contiene las varianzas de cada variable, mientras que los valores fuera de la diagonal son las covarianzas entre estas (Figura 3).

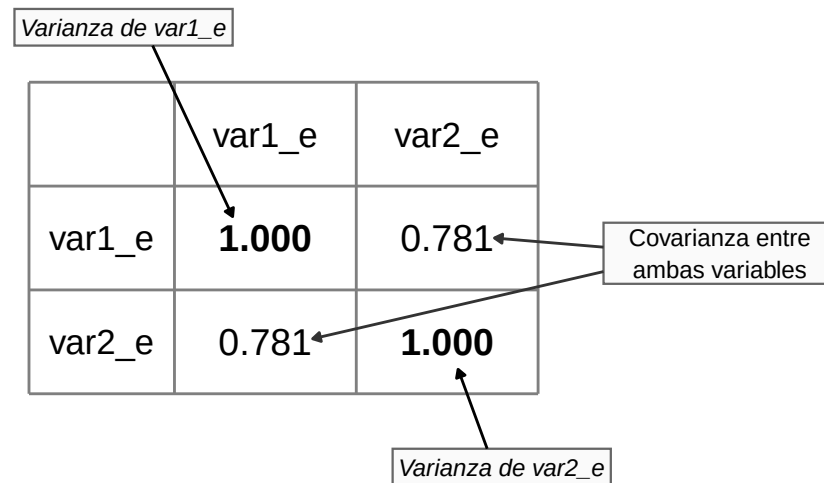


Figure 4: Matriz de covarianza para dos variables

La covarianza nos dice de qué forma los datos se “mueven” juntos. En este caso cuando los valores de `var1` aumentan también los hacen los valores de `var2` (hay una correlación positiva), lo que también se observó en la gráfica de dispersión.

Un punto importante es que al calcular la matriz de covarianza con variables centradas y escaladas lo que también hacemos es calcular la matriz de correlación entre las mismas.

Vectores propios y valores propios de la matriz de covarianza

Para encontrar los CP , que previamente definimos como combinaciones lineales de las variables originales que capturan la mayor varianza posible, necesitamos descomponer la matriz de covarianza, con lo que obtendremos los vectores y valores propios de la misma. Para esto debemos resolver la ecuación característica:

$$C\vec{v} = \lambda\vec{v} \quad (4)$$

Donde C es la matriz de covarianza, $\vec{v}$ es un vector propio y λ es un valor propio. Para resolver lo anterior despejamos el lado izquierdo de la ecuación:

$$C\vec{v} - \lambda\vec{v} = \vec{0} \quad (5)$$

Si factorizamos $\vec{v}$:

$$(C - \lambda I)\vec{v} = \vec{0} \quad (6)$$

Queremos encontrar vectores que no sean cero, por lo que debemos resolver el determinante:

$$\det(C - \lambda I) = 0 \quad (7)$$

Al desarrollar lo anterior definimos un polinomio característico, cuyas soluciones serán los valores propios que satisfacen nuestra ecuación inicial $C\vec{v} = \lambda\vec{v}$. Finalmente, para encontrar cada $\vec{v}$ resolvemos el sistema resultante $(C - \lambda I)\vec{v} = \vec{0}$, donde $\vec{v} = (x, y)$. Con este enfoque podemos establecer que habrá tantos valores y vectores propios como variables haya en nuestros datos originales.

De forma general:

- Los vectores propios indican las direcciones en las que los datos tienen la mayor dispersión (estos son propiamente los CP).
- Los valores propios indican cuánta varianza (o variación) hay en cada una de esas direcciones.

Para encontrar los vectores y valores propios de nuestra matriz de covarianza es posible utilizar la función dedicada `eigen()`:

```
# Usar eigen() para obtener vectores y valores propios
cov_e <- eigen(cov_m)

# Guardar por separado vectores propios
vecp_e <- cov_e$vectors

# Guardar por separado valores propios
valp_e <- cov_e$values
```

En el resultado de la función `eigen()` los valores propios (`$values`) se muestran de mayor a menor, mientras que los vectores propios (`$vectors`) se muestran como una matriz donde cada columna es un vector individual.

Cada vector propio, y su representación visual, puede considerarse como una “línea” que captura la mayor variación en los datos originales (Bro & Smilde, 2014), lo que puede ilustrarse directamente con una gráfica:

```

# Primer vector propio
vp1 <- vecp_e[,1]

# Pendiente del primer vector propio
vp1_m <- vp1[2] / vp1[1]

# Segundo vector propio
vp2 <- vecp_e[,2]

# Pendiente del segundo eigenvector
vp2_m <- vp2[2] / vp2[1]

# Gráfico de dispersión mostrando la representación de cada vector propio
plot_data <- ggplot(data.frame(datos_estandarizados), aes(x = var1_e, y = var2_e)) +
  geom_point(color = "#EE5C42", size = 2) +
  geom_vline(xintercept = 0, linewidth = .5) +
  geom_hline(yintercept = 0, linewidth = .5) +
  geom_abline(slope = vp1_m, color = "blue", linewidth = 0.7) +
  geom_abline(slope = vp2_m, color = "red", linewidth = 0.7) +
  xlab("Variable 1 estandarizada") +
  ylab("Variable 2 estandarizada") +
  theme_classic()

plot_data

```

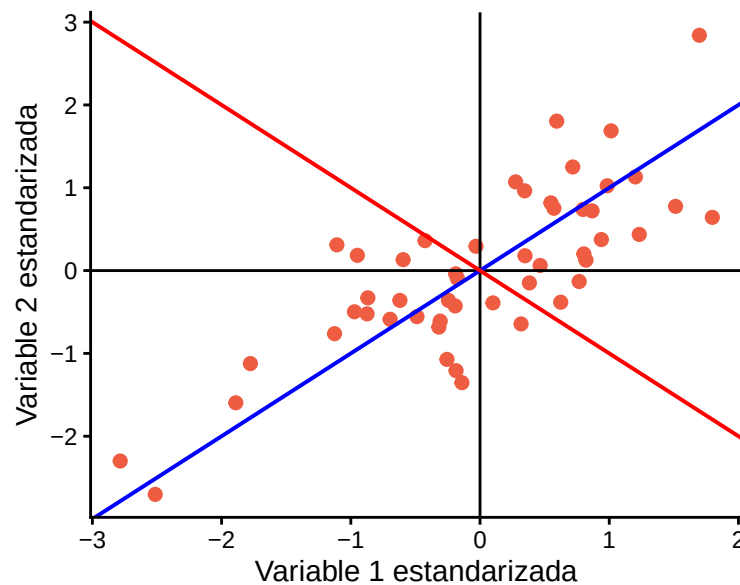


Figure 5: Representación de los vectores propios sobre el espacio bidimensional estandarizado; la línea azul denota la dirección del primer componente principal (máxima dispersión) y la línea roja el segundo componente (ortogonal al primero).

En la Figura 4 la línea azul representa al primer vector propio (o primer componente CP_1), mientras que la línea roja al segundo (o segundo componente CP_2).

Como mencionamos antes, los vectores propios son ortogonales o perpendiculares (forman un ángulo de 90°):


```
# Multiplicar ambos vectores propios (producto punto)
vp1 %*% vp2
```

```
##      [,1]
## [1,]    0
```

En cuanto a los valores propios, puede demostrarse que sus valores numéricos son iguales a la varianza de los puntos de datos proyectados en el componente principal correspondiente (es decir, la suma de los cuadrados de las distancias dividida entre $n - 1$):

$$\lambda_j = \text{Var}(Y_j) = \frac{1}{n-1} \sum_{i=1}^n y_{ij}^2 \quad (8)$$

Donde y_{ij} representa la puntuación o coordenada (*score*) de la muestra i proyectada sobre el componente principal j . Al estar los datos previamente centrados, cada valor y_{ij} equivale matemáticamente a la distancia del punto respecto al origen, por lo que la ecuación anterior representa la suma de los cuadrados de las distancias dividida entre $n - 1$.

Porcentajes de variación explicada

Los valores propios representan las varianzas de los componentes principales. La suma de todas las varianzas (la varianza o variación total) se puede utilizar para calcular el porcentaje de variación de cada valor propio y posteriormente visualizar dichos porcentajes con una gráfica de barras:

```
# Calcular el porcentaje de variación explicada de cada valor propio
pvar_vp <- (valp_e * 100) / sum(valp_e)

# Data frame con porcentajes de varianza
var_per <- data.frame(
  CP = c("CP1", "CP2"),
  PV = pvar_vp
)

# Gráfico de sedimentación
ggplot(var_per, aes(x = CP, y = PV)) +
  geom_col(width = 0.5, color = "black") +
  xlab("Componente Principal") +
  ylab("Porcentaje de variación explicada (%)") +
  theme_classic()
```

Como puede observarse en la Figura 5 el primer componente “atrapa” la mayor parte de la variación observada en nuestros datos simulados. Esto resulta más claro si regresamos a la Figura 4 y observamos la línea azul (CP_1) en comparación con la línea roja (CP_2). Notaremos que la mayoría de los datos “siguen” la tendencia marcada por CP_1 mientras que no lo hacen tanto para CP_2 .

Cargas

Los vectores propios calculados por la función `eigen()` están normalizados. Como se mencionó previamente esto significa que su norma o longitud es igual a 1:

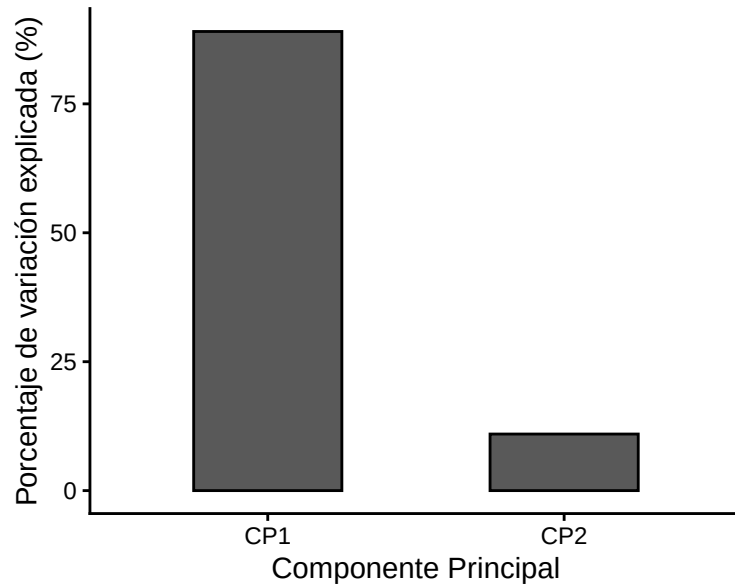


Figure 6: Gráfica de barras para el modelo bidimensional, ilustrando el porcentaje absoluto de varianza capturado por cada uno de los dos componentes principales.

```
# Norma del primer eigenvector
norm(as.matrix(vp1), "F")
```

```
## [1] 1
```

```
# Norma del segundo eigenvector
norm(as.matrix(vp2), "F")
```

```
## [1] 1
```

También se mencionó que los componentes de cada vector propio también se denominan cargas (*loadings*) y pueden interpretarse como la contribución de cada variable al componente principal correspondiente o, más estrictamente, como los coeficientes de la combinación lineal de las variables originales a partir de las cuales se construyen los componentes principales. El signo en estos valores indica la dirección en que la variable contribuye al componente.

Es posible generar un marco de datos con estos valores y observar las contribuciones de cada variable a cada componente principal:

```
# Marco de datos con ambos vectores propios
loads <- data.frame(
  VAR = c("var_1", "var_2"),
  CP1 = round(vp1, 2), # Primer vector propio
  CP2 = round(vp2, 2)  # Segundo vector propio
)
```

```
loads
```

```
##      VAR  CP1  CP2
```

```
## 1 var_1 0.71 -0.71
## 2 var_2 0.71  0.71
```

En este caso las contribuciones absolutas de cada variable son idénticas, ya que solo se trata de dos variables. Sin embargo, no perdamos de vista que lo anterior puede ser útil en datos con muchas dimensiones, donde se obtendrán valores de cargas distintos y se podrá establecer cuáles variables causan las agrupaciones o diferencias en el gráfico de puntuaciones del *ACP*.

Representar los datos en dimensiones menores

Si cambiamos la base de los datos originales a la indicada por los vectores propios, lo que hacemos de forma directa es “rotar” los datos de tal forma que los componentes pasan a ser los nuevos ejes de coordenadas. Para esto multiplicamos la matriz con los datos estandarizados por la matriz de vectores propios obtenida al descomponer la matriz de covarianza:

```
# Cambiar la base de los datos originales
puntuaciones_acp <- datos_estandarizados %*% vecp_e
```

Podemos comparar la Figura 4 con una gráfica de las `puntuaciones_acp` para tener una idea de cómo se han rotado los datos una vez que cambiamos la base:

```
# Gráfico de dispersión con los datos rotados
plot_rotation <- ggplot(data.frame(puntuaciones_acp), aes(X1, X2)) +
  geom_point(color = "#EE5C42", size = 2) +
  geom_vline(xintercept = 0, linewidth = .5, color = "red") +
  geom_hline(yintercept = 0, linewidth = .5, color = "blue") +
  xlab(paste0("CP1 (", round(var_per$PV[1], 2), "%)")) +
  ylab(paste0("CP2 (", round(var_per$PV[2], 2), "%)")) +
  ylim(c(-5, 5)) +
  theme_classic()

# Ambos gráficos lado a lado
ggarrange(plot_data, plot_rotation)
```

Dado que el CP_1 explica la mayor parte de la variación en los datos, podemos omitir el CP_2 y representar cada punto en una sola dimensión, aquí como puntos verdes:

```
# Puntos de datos solo del CP1
datos_pc1 <- data.frame(v1 = puntuaciones_acp[,1], v2 = rep(0, nrow(puntuaciones_acp)))

# Gráfico de dispersión mostrando los puntos proyectados del CP1 (puntos azules)
plot_rotation +
  geom_point(data = datos_pc1, aes(v1, v2), color = "green", size = 2) +
  ylim(c(-1.8, 1.8))
```

Notemos en la Figura 7 que cada punto verde es la “proyección” de cada punto naranja (en dos dimensiones) sobre CP_1 . Lo anterior es el razonamiento de partida para generalizar lo aplicado en esta sección a datos con más dimensiones: si solo unos cuantos componentes “atrapan” la mayor parte de la variación en los datos originales, es posible descartar el resto y representar el conjunto en un menor número de dimensiones.

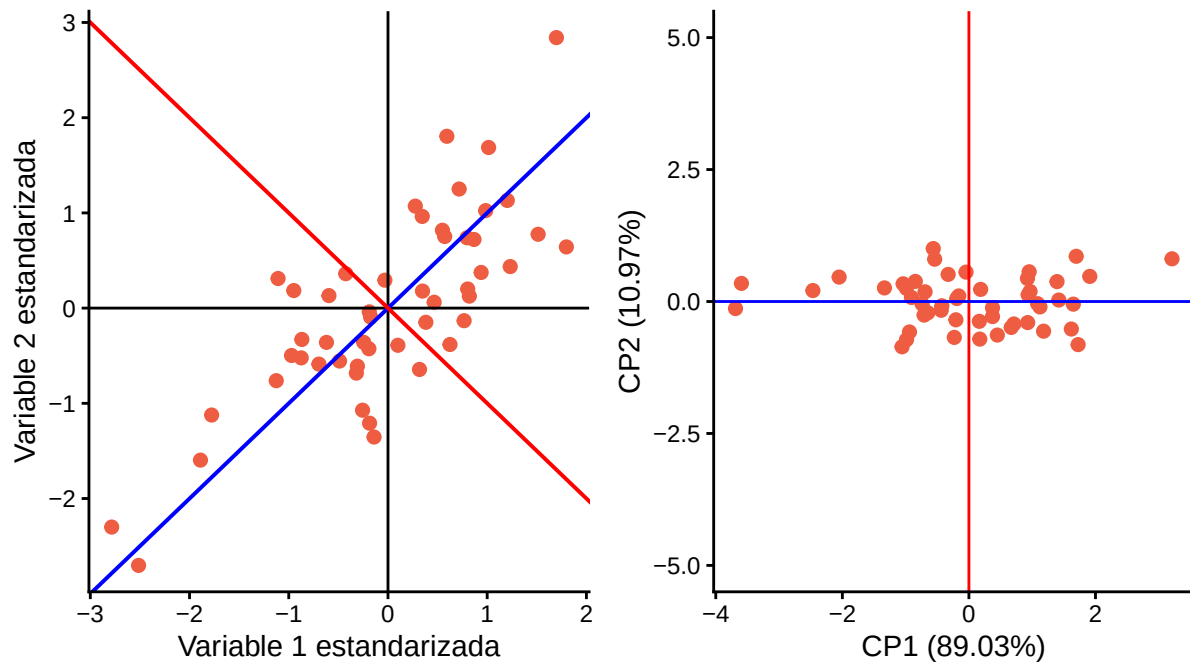


Figure 7: Comparación del espacio geométrico de los datos simulados: a la izquierda se presentan las variables estandarizadas en sus ejes originales y a la derecha se observa la rotación lograda tras el cambio de base.

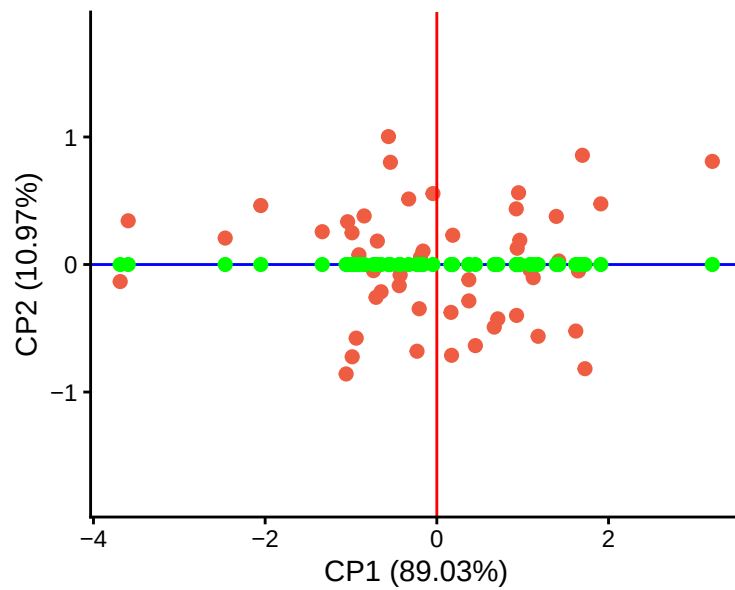


Figure 8: Proyección ortogonal de los datos bidimensionales simulados sobre el eje unidimensional de $CP1$ (puntos azules)

APLICACIÓN DEL ACP A LAS CIENCIAS QUÍMICAS

Breve marco de referencia - Análisis químico de vinos

El vino es una bebida alcohólica cuya composición se puede considerar como altamente compleja ya que contiene cientos de compuestos químicos de naturaleza volátil y no volátil. De forma más específica, el vino se define como una solución acuosa que contiene principalmente agua (70-90% p/p) y etanol (8-20% v/v), junto con concentraciones variables de polioles como el glicerol, azúcares reductores y ácidos orgánicos. Además, la identidad sensorial y calidad comercial de la bebida reside en sus constituyentes minoritarios: compuestos fenólicos (antocianinas, taninos, flavonoles, etc.), aminoácidos y compuestos volátiles traza (en muy pequeña cantidad) tales como ésteres de acetato y terpenos.(Fernández et al., 2009)

Con el objetivo de caracterizar la composición química de vinos de forma detallada, se requieren aplicar distintos métodos analíticos tales como la cromatografía de gases acoplada a espectrometría de masas (*CG-MS*) y la espectroscopia infrarroja con transformada de Fourier (*FT-IR*). La *CG-MS* se enfoca en la identificación y cuantificación de compuestos volátiles. El principio básico de la *CG-MS* se basa en la inyección, en el cromatógrafo de gases, de una muestra que es vaporizada y arrastrada por un gas portador inerte como el helio (fase móvil) a través de una columna capilar recubierta internamente por un polímero de alta estabilidad térmica (fase estacionaria). La separación física de los analitos ocurre en función de su presión de vapor y su coeficiente de partición entre ambas fases mientras se ejecuta un programa de aumento controlado de temperatura en un horno de precisión. Al eluir de la columna, los compuestos individuales ingresan al espectrómetro de masas, donde se someten a ionización por impacto electrónico a una energía estandarizada. Esta ionización fragmenta las moléculas en un patrón espectral de iones característico según su relación masa-carga, permitiendo identificar compuestos desconocidos mediante el índice de retención cromatográfico y la concordancia de los espectros de fragmentación con librerías de referencia como la NIST.(Pati et al., 2021)

Por otra parte, el análisis de muestras mediante *FT-IR* se basa en la interacción de la radiación electromagnética del infrarrojo medio (región MIR, de 4000 a 400 cm^{-1}) con los compuestos de la muestra. Al absorber energía en estas longitudes de onda, los enlaces covalentes de los compuestos orgánicos experimentan transiciones vibracionales (modos de tensión y flexión), cuya intensidad de absorción es directamente proporcional a la fuerza de enlace y a la masa de los átomos involucrados. El acoplamiento de accesorios de reflexión total atenuada (*ATR*), basados en la reflexión interna de un haz de luz dentro de un cristal de alto índice de refracción (como el diamante) en contacto con el vino, permite el análisis directo del líquido sin preparación previa. Esto genera un espectro de absorción que se considera como una “huella digital” única para cada muestra y que a su vez es un reflejo de la composición intrínseca de los polifenoles, azúcares, ácidos orgánicos y alcoholes. Particularmente, la asignación de picos en el espectro infrarrojo del vino permite asociar diferentes bandas con familias de compuestos específicos útiles para la caracterización.(Grijalva-Verdugo et al., 2018)

En conjunto los métodos de análisis descritos generan una gran cantidad de datos. Para abordar el análisis de estos el *ACP* constituye una metodología exploratoria que permite establecer correlaciones y posibles diferencias entre muestras de vino de acuerdo a su origen, factores biológicos, climáticos, geográficos y tecnológicos.

Paquetes

Si se pasa directamente de la primera sección a esta parte del tutorial es recomendable reiniciar la sesión de R, lo que evitará que se sature con todos los datos y variables definidas en dicha sección.

Los paquetes recomendados para esta sección son los mismos. Además, se sugiere el uso del paquete `readr` para poder descargar directamente los datos de vino en su sesión de R (se requiere de conexión a internet), así como `purrr` para poder realizar cálculos simultáneos sobre columnas del conjunto de datos.

```
# Cargar los paquetes en su sesión de R
library(ggplot2) # Gráficas
library(ggpubr)  # Gráficas conjuntas
library(dplyr)   # Manipulación de datos
library(readr)   # Importación o descarga de datos
library(purrr)   # Programación funcional para operaciones sobre columnas
library(stringr) # Manipulación de cadenas de caracteres
```

Datos de muestras de vino analizadas obtenidos mediante GC-MS y FT-IR

Los datos utilizados en esta parte del tutorial son de vinos provenientes de Argentina, Chile, Australia y Sudáfrica. (Bro & Smilde, 2014; Svok et al., 2008)

Los datos a importar son los siguientes:

- `Label_Wine_samples.csv`: Tipos o regiones de cada muestra de vino analizada.
- `Label_Pred_values_IR.csv`: Nombre de las variables obtenidas o predichas a partir de los espectros FT-IR de las muestras, lo que incluye acidez, concentración de azúcares, pH, concentración de etanol, entre otras.
- `Pred_values.csv`: Valores o niveles de cada una de las variables obtenidas mediante FT-IR.
- `Label_Aroma_comp.csv`: Nombre de los compuestos volátiles identificados y cuantificados mediante GC-MS en todas las muestras de vino.
- `Aroma_compounds.csv`: Niveles o cantidades de cada compuesto volátil en las muestras de vino analizadas.

La función `read_csv()` permite la descarga directa de datos alojados en repositorios en línea, en este caso *GitHub* (se recomienda no modificar los enlaces mostrados a continuación al utilizar dicha función). Con los datos listados construiremos un solo marco de datos para poder analizarlo mediante *ACP*. Primero los datos correspondientes a *FT-IR*:

```
# Nombres de las variables obtenidas mediante FT-IR
var_names_ftir <- read_csv(
  "https://raw.githubusercontent.com/juanpa-ch/PCA-CIENCIAS-QUIMICAS-DATOS/refs/heads/main/Label_Pred_v"
var_names_ftir <- names(var_names_ftir)

# Conjunto de valores obtenidos para cada variable en cada muestra de vino
datos_ftir <- read_csv(
  "https://raw.githubusercontent.com/juanpa-ch/PCA-CIENCIAS-QUIMICAS-DATOS/refs/heads/main/Pred_values."
  col_names = var_names_ftir
)
```

Ahora repetamos lo anterior para los datos obtenidos mediante *CG-MS*:

```
# Nombres de las variables obtenidas mediante CG-MS
var_names_cgms <- read_csv(
  "https://raw.githubusercontent.com/juanpa-ch/PCA-CIENCIAS-QUIMICAS-DATOS/refs/heads/main/Label_Aroma_"
)
var_names_cgms <- names(var_names_cgms)

# Conjunto de valores obtenidos para cada compuesto en cada muestra de vino
datos_cgms <- read_csv(
  "https://raw.githubusercontent.com/juanpa-ch/PCA-CIENCIAS-QUIMICAS-DATOS/refs/heads/main/Aroma_compou"
```

```
col_names = var_names_cgms
)
```

Por último, unamos ambos conjuntos de datos:

```
# Conjunto de datos de vino (FT-IR más CG-MS)
datos_vino <- cbind(datos_ftir, datos_cgms)

# Cambiar los nombres duplicados de "Ethanol" en orden de aparición
colnames(datos_vino)[colnames(datos_vino) == "Ethanol"] <- c("Ethanol_FTIR", "Ethanol_CGMS")
```

Al conjunto anterior también le podemos agregar una columna extra para especificar las muestras de vino correspondientes a cada renglón:

```
# Etiquetas o tipos de vino
muestra_vino <- read_csv(
  "https://raw.githubusercontent.com/juanpa-ch/PCA-CIENCIAS-QUIMICAS-DATOS/refs/heads/main/Label_Wine_s
  col_names = FALSE
)
names(muestra_vino) <- "Muestra_Vino"

datos_vino <- cbind(muestra_vino, datos_vino)
```

Para mayor claridad y comprensión, se recomienda ejecutar `View(datos_vino)` directamente en la consola de *R* para poder visualizar el conjunto de datos completo.

Con lo anterior cada muestra de vino queda asociada a sus valores respectivos de etanol, acidez, pH, glicerol, metanol, etc. Sin embargo, para el *ACP* no es necesario indicar de forma explícita a qué muestra corresponde cada fila, por lo que omitiremos la columna `vino_Sample` para los cálculos y el análisis. Solo no perdamos de vista que cada renglón corresponde a una muestra o tipo de vino. Dicha información también se retomará cuando hagamos gráficas con los resultados del *ACP*.

Otra recomendación importante es tomar en cuenta la estructura del marco de datos, en esta caso las muestras de vino están indicadas en cada renglón y las variables respuesta en cada columna. Esto es importante al calcular la matriz de covarianza o usar funciones dedicadas para realizar el *ACP*, ya que se debe definir si se quiere reducir las dimensiones respecto a las filas o respecto a las columnas. Aquí estamos interesados en reducir el número de dimensiones respecto a las variables respuesta (columnas) y posteriormente identificar si existen similitudes o diferencias entre las muestras de vino (filas) según su origen u otra característica.

Estandarizar las variables - Centrar y dividir por la desviación estándar

Primero calculemos las medias y desviaciones estándar de cada una de las variables medidas en las muestras de vino:

```
# Omitir la columna correspondiente a los códigos de cada muestra de vino
datos_vino_sn <- datos_vino %>% select(-Muestra_Vino)

# Medias de cada variable
var_means <- unlist(map(datos_vino_sn, mean))

# Desviación estándar de cada variable
var_sd <- unlist(map(datos_vino_sn, sd))
```

Como siguiente paso utilizemos la función `map2()` para restar las medias calculadas en el paso anterior a su columna correspondiente, con esto los datos quedarán centrados:

```
# Centrar cada variable (restar la media)
datos_vino_acp <- map2(
  datos_vino_sn, var_means, .f = function(x, mean) x - mean
)
```

Notemos que para utilizar `map2()` se requiere especificar una función como tercer argumento. En este caso con dicha función especificamos que a cada columna del conjunto de datos original se le reste la media correspondiente. El resultado, una lista, lo guardamos por separado en la variable `datos_vino_acp`.

Ahora que las variables están centradas, dividamos entre la desviación estándar de cada variable para estandarizar cada una. Recordemos que dividir por las desviaciones estándar es una forma de dar a cada variable la misma importancia a pesar de su rango, magnitud y/o escala de medición:

```
# Dividir por la desviación estándar de cada variable
datos_vino_acp <- map2(
  datos_vino_acp, var_sd, .f = function(x, sd) x / sd
)
```

Para facilitar su uso transformamos la lista anterior en un marco de datos. Recordemos que podemos visualizarlos si ejecutamos `View(datos_vino_acp)`:

```
# Crear un marco de datos a partir de la lista anterior
datos_vino_acp <- data.frame(datos_vino_acp)
```

En estos datos cada fila corresponde a la misma muestra de vino que en los datos originales.

Calcular la matriz de covarianza

De forma similar al ejemplo con dos dimensiones, es posible utilizar la función `cov()` para obtener la matriz de covarianza:

```
# Calcular la matriz de covarianza
cov_vino <- cov(datos_vino_acp)
```

Si visualiza esta matriz con `View(cov_vino)` los valores de la diagonal son los valores de varianzas de cada variable y los valores fuera de la diagonal son las covarianzas (o correlaciones) entre variables.

Obtener los valores y vectores propios de la matriz de covarianza

Ahora utilizamos la función `eigen()` para obtener los valores y vectores propios de la matriz de covarianza correspondiente a los datos de vino:

```
# eigen() para obtener valores y vectores propios
eg_vino <- eigen(cov_vino)

# Valores propios
valp_vino <- eg_vino$values

# Vectores propios
vecp_vino <- eg_vino$vectors
```


Recordemos que con este procedimiento el número de valores y vectores propios es igual al número de variables original:

```
# Número de valores propios
length(valp_vino)
```

```
## [1] 71
```

```
# Número de vectores propios
ncol(vecp_vino)
```

```
## [1] 71
```

```
# Número de variables en datos originales
ncol(datos_vino_sn)
```

```
## [1] 71
```

Porcentajes de variación explicada

Con los valores propios ahora calculemos el porcentaje de variación de cada *CP* tomando en cuenta la suma total de las varianzas (especificada por los valores propios):

```
# Calcular el porcentaje de variación de cada componente principal
p_var <- valp_vino * 100 / sum(valp_vino)

# Marco de datos con los porcentajes de variación
pvar_cp <- data.frame(
  CP = 1:71,          #Número de componente
  PER = round(p_var, 4) #Porcentaje de variación
)
```

Si visualizamos el conjunto de datos anterior con `View(pvar_cp)`, notaremos que un gran número de componentes tienen porcentajes de variación por debajo del 5%. De hecho si calculamos el porcentaje de variación acumulado para los primeros 14 componentes, veremos que estos abarcan más del 80% de la variación total de los datos:

```
sum(p_var[1:14])
```

```
## [1] 84.60537
```

Con esto en mente realicemos una gráfica de barras que solo represente el porcentaje de variación de estos 14 componentes. De esta forma también evitaremos que nuestra gráfica parezca muy saturada al intentar graficar la variación de 71 componentes:

```
# Gráfico de barras
ggplot(
  pvar_cp[1:14, ],
  aes(x = reorder(CP, order(PER, decreasing = TRUE)), y = PER)
) +
```

```
geom_col(width = 0.5, color = "black") +
xlab("Componente Principal") +
ylab("Porcentaje de variación (%)") +
theme_classic()
```

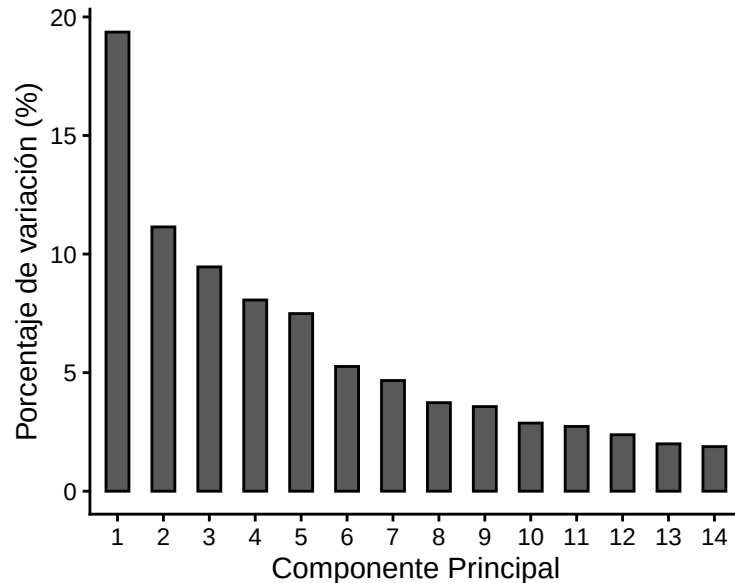


Figure 9: Gráfica de barras con la variación de los primeros 14 componentes principales.

El hecho de que la mayoría de los componentes tengan porcentajes de variación bajos se puede usar como criterio para eliminarlos y representar las nuevas variables en un menor número de dimensiones. También es importante mencionar que hay criterios concretos para descartar componentes:

- La regla del codo: Observar visualmente el gráfico de barras e identificar el punto de inflexión donde la caída de la varianza explicada se vuelve drásticamente más plana o marginal. En la Figura 9 es posible observar este tipo de caída después de CP_5 .
- El criterio de *Kaiser*: Seleccionar únicamente los componentes cuyos valores propios sean mayores a 1 (un componente con un valor propio menor a 1 explica menos varianza que una sola variable estandarizada):

```
valp_vino[1:20]
```

```
## [1] 13.7462027  7.9126478  6.7158062  5.7243968  5.3172228  3.7342520
## [7]  3.3134855  2.6509381  2.5332034  2.0395649  1.9385510  1.6901070
## [13]  1.4183264  1.3351078  1.1993986  1.0295915  0.9391402  0.8594141
## [19]  0.7884762  0.7317485
```

Notemos como a partir del valor propio 17 (contando los valores de izquierda a derecha) las magnitudes caen por debajo de 1.

Representar los datos en dimensiones menores

Idealmente, si el CP_1 y el CP_2 reunieran la mayor parte de la variación, digamos más del 90%, sería posible hacer una buena representación de los datos en un gráfico de dispersión de dos dimensiones. En este caso

los datos se alejan del caso ideal, pero aún así podemos intentar representar los datos en gráficas de dos dimensiones para buscar posibles relaciones o diferencias entre las muestras de vino. Para nuestros datos los primeros cuatro CP representan alrededor del 50% de la variación. De esta forma podemos intentar detectar grupos o diferencias creando dos gráficas de dispersión: la primera con CP_1 y CP_2 , y la segunda con CP_3 y CP_4 .

Primero, cambiemos la base de los datos transformados a la establecida por los vectores propios:

```
# Cambiar la base de los datos estandarizados
puntuaciones_vino_acp <- as.matrix(datos_vino_acp) %*% vecp_vino

# Transformar en un marco de datos
puntuaciones_vino_acp <- data.frame(datos_vino_acp)

# Cambiar el nombre de las columnas para señalar cada componente principal
colnames(puntuaciones_vino_acp) <- paste0("CP", pvar_cp$CP)
```

Segundo, añadamos una columna para señalar el origen geográfico de cada muestra de vino. Para esto utilizemos la información en la columna `Muestra_Vino` de los datos originales:

```
# Añadir una columna con el origen de cada muestra de vino
puntuaciones_vino_acp <- puntuaciones_vino_acp %>%
  mutate(
    Origen = str_sub(datos_vino$Muestra_Vino, start = 1, end = 3)
  ) %>%
  relocate(Origen)
```

Si ejecutamos `View(puntuaciones_vino_acp)` veremos que la primera columna corresponde al origen de cada muestra de vino, donde *ARG* se refiere Argentina, *CHI* a Chile, *AUS* a Australia y *SOU* a Sudáfrica.

Con el conjunto de datos anterior hagamos ambos gráficos de dispersión tomando solo las puntuaciones de CP_1 , CP_2 , CP_3 y CP_4 .

```
# Gráfico de dispersión para CP1 y CP2
cp12 <- ggplot(
  puntuaciones_vino_acp,
  aes(CP1, CP2, color = Origen, shape = Origen)
) +
  geom_point(size = 3) +
  scale_color_discrete(
    name = "Origen",
    labels = c("Argentina", "Australia", "Chile", "Sudáfrica")
  ) +
  scale_shape_discrete(
    name = "Origen",
    labels = c("Argentina", "Australia", "Chile", "Sudáfrica")
  ) +
  xlab(paste0("CP1 (", round(pvar_cp$PER[1], 1), "%)")) +
  ylab(paste0("CP2 (", round(pvar_cp$PER[2], 1), "%)")) +
  theme_classic() +
  theme(legend.position = "none")

# Gráfico de dispersión para CP3 y CP4
cp34 <- ggplot(
```

```

puntuaciones_vino_acp,
aes(CP3, CP4, color = Origen, shape = Origen)
) +
geom_point(size = 3) +
scale_color_discrete(
  name = "Origen",
  labels = c("Argentina", "Australia", "Chile", "Sudáfrica")
) +
scale_shape_discrete(
  name = "Origen",
  labels = c("Argentina", "Australia", "Chile", "Sudáfrica")
) +
xlab(paste0("CP3 (", round(pvar_cp$PER[3], 1), "%)")) +
ylab(paste0("CP4 (", round(pvar_cp$PER[4], 1), "%)")) +
theme_classic()

# Ambas gráficas lado a lado
ggarrange(cp12, cp34, common.legend = TRUE, legend = "bottom", labels = "AUTO")

```

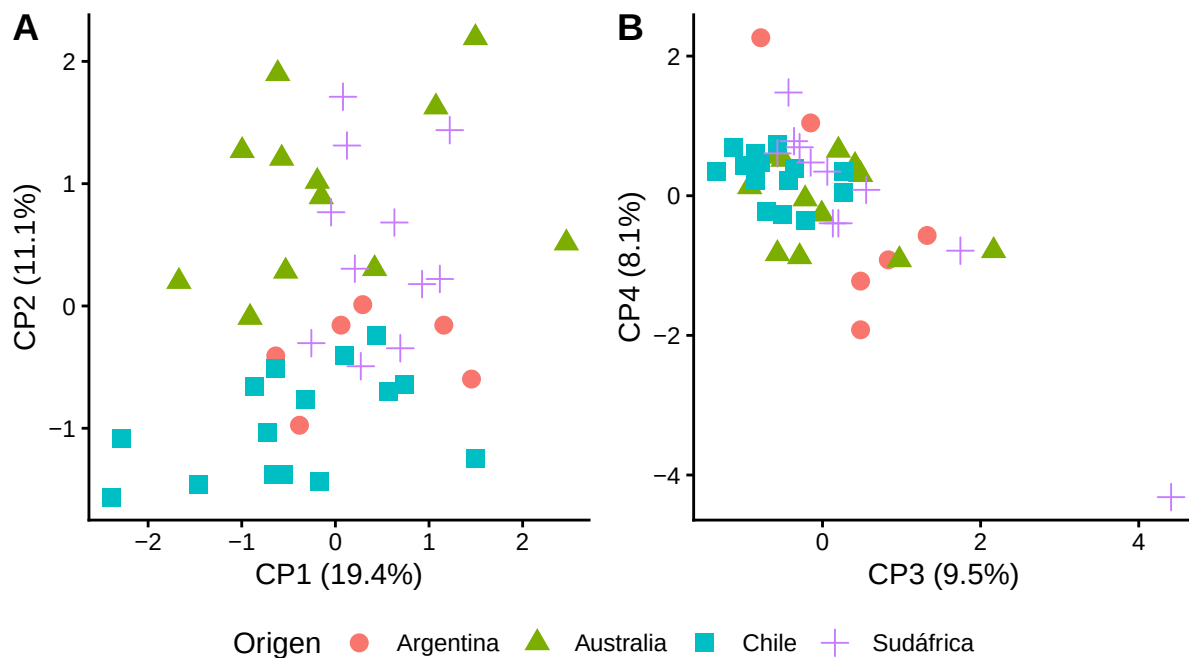


Figure 10: Gráficas de puntuaciones correspondientes al segundo componente principal frente al primero (A) y al cuarto componente frente al tercero (B).

Cargas

Como se mencionó anteriormente, los elementos de cada vector propio representan la carga (o importancia) de cada variable en el componente principal correspondiente. Estos valores también pueden representarse en gráficos de dos dimensiones. Primero hagamos un marco de datos con las cargas que obtuvimos de los datos de vino y añadamos un columna para señalar el nombre de cada variable:

```

# Marco de datos con puntuaciones de carga
loads_vino <- data.frame(vecp_vino)
colnames.loads_vino <- colnames(puntuaciones_vino_acp)[2:72]

# Columna con el nombre de las variables originales
loads_vino <- loads_vino %>%
  mutate(Variable = colnames(datos_vino_sn)) %>%
  relocate(Variable)

```

Si visualizamos el marco de datos con `View.loads_vino` notaremos que cada valor numérico señala la carga de la variable correspondiente sobre cada uno de los 71 *CP*.

Podemos generar un gráfico de dispersión con estos valores, lo que a su vez puede ayudar a explicar cuáles variables generan los agrupamientos o diferencias en la Figura 9. También tomemos en cuenta solo los primeros cuatro *CP* y para evitar la saturación de nuestra gráfica con los nombres de todas las variables, filtremos las cargas estableciendo un valor absoluto de 0.2 y utilicemos un paquete especial (`ggrepel`) para evitar los traslapes entre nombres de variables. Primero, ejecutemos `install.packages("ggrepel")` para instalar dicho paquete, si no está instalado, y posteriormente reproduzcamos el siguiente código:

```

# Cargar ggrepel
library(ggrepel)

# Filtrar solo las variables con cargas importantes para CP1 y CP2
loads_cp12 <- subset.loads_vino, abs(CP1) > 0.2 | abs(CP2) > 0.2)

# Graficar el marco de datos filtrado
ld_cp12 <- ggplot.loads_cp12, aes(CP1, CP2)) +
  # Puntos de las variables filtradas
  geom_point(color = "blue", size = 2.5) +

  # Líneas de referencia discontinuas en los ejes para limpiar el fondo
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray60") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray60") +

  # Etiquetas inteligentes que no se traslapan
  geom_text_repel(
    aes(label = Variable),
    size = 3.8,
    max.overlaps = Inf,
    box.padding = 0.4,
    point.padding = 0.3,
    segment.color = "gray50",
    segment.size = 0.3
  ) +

  # Títulos y etiquetas de los ejes
  xlim(c(-0.4, 0.4)) +
  ylim(c(-0.4, 0.4)) +
  xlab(paste0("CP1 (", round(pvar_cp$PER[1], 1), "%)")) +
  ylab(paste0("CP2 (", round(pvar_cp$PER[2], 1), "%)")) +
  theme_classic()

ld_cp12

```

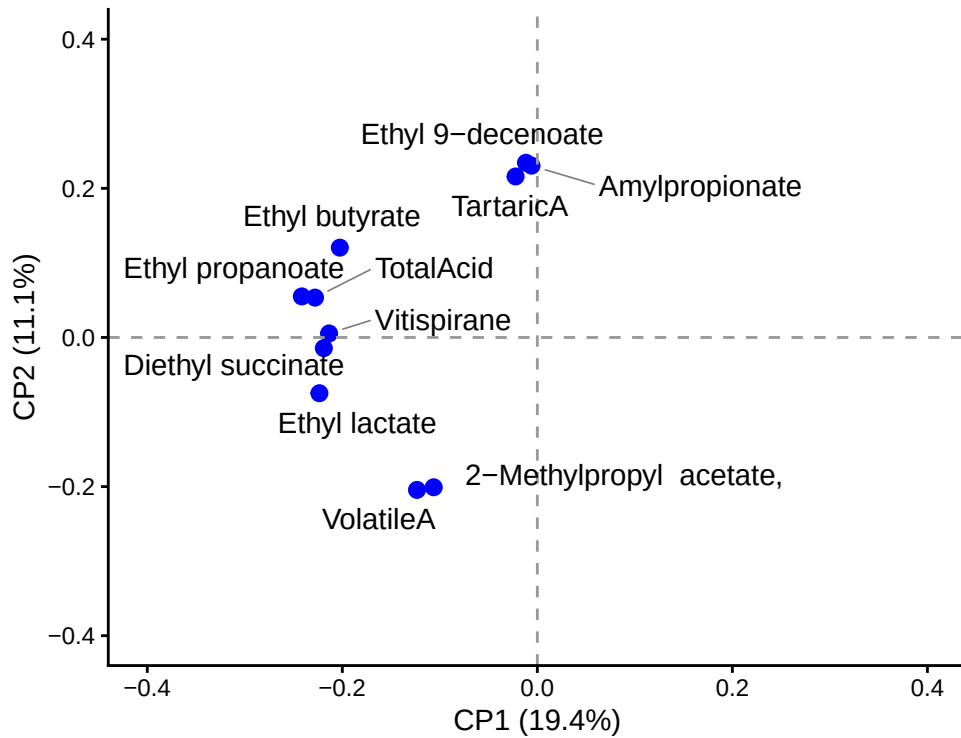


Figure 11: Gráfica de cargas con un valor absoluto mayor a 0.2 correspondiente el plano bidimensional de CP_1 y CP_2 .

En la Figura 10 se observan las variables que se pueden considerar más importantes para CP_1 y CP_2 . De forma similar podemos adecuar el código anterior para el caso de CP_3 y CP_4 :

```
# Filtrar solo las variables con cargas importantes para CP3 y CP4
loads_cp34 <- subset(loads_vino, abs(CP3) > 0.2 | abs(CP4) > 0.2)

# Graficar el marco de datos filtrado
ld_cp34 <- ggplot(loads_cp34, aes(CP3, CP4)) +
  # Puntos de las variables filtradas
  geom_point(color = "blue", size = 2.5) +

  # Líneas de referencia discontinuas en los ejes para limpiar el fondo
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray60") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray60") +

  # Etiquetas inteligentes que no se traslapan
  geom_text_repel(
    aes(label = Variable),
    size = 3.8,
    max.overlaps = Inf,
    box.padding = 0.4,
    point.padding = 0.3,
    segment.color = "gray50",
    segment.size = 0.3
  ) +
```

```
# Títulos y etiquetas de los ejes
xlim(c(-0.4, 0.4)) +
ylim(c(-0.4, 0.4)) +
xlab(paste0("CP3 (", round(pvar_cp$PER[3], 1), "%)") +
ylab(paste0("CP4 (", round(pvar_cp$PER[4], 1), "%)") +
theme_classic()

ld_cp34
```

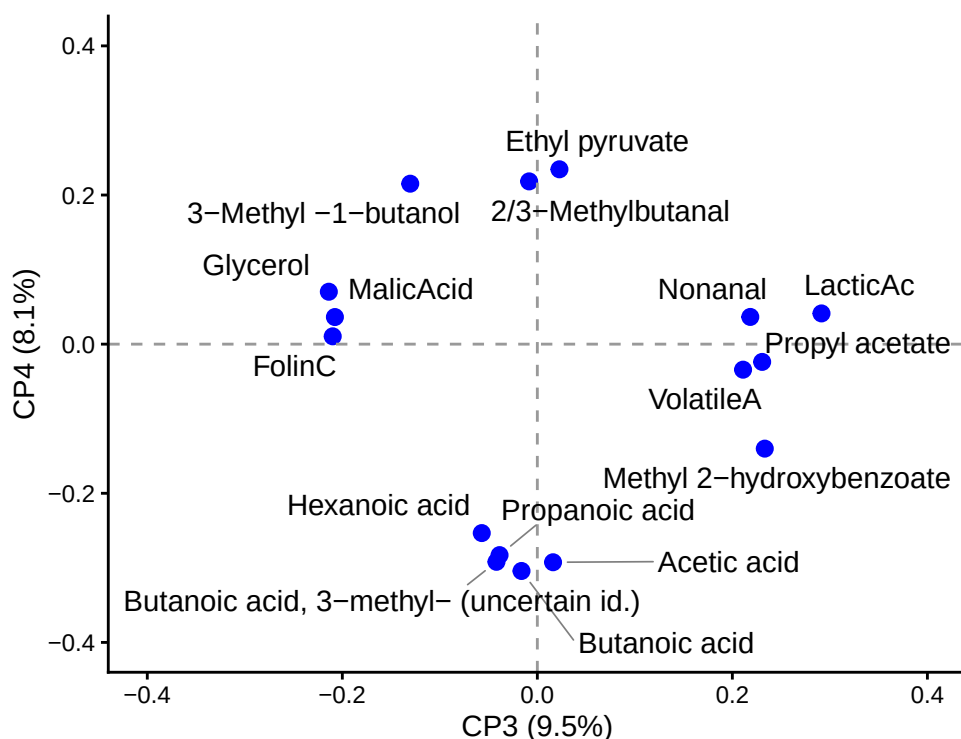


Figure 12: Gráfica de cargas con un valor absoluto mayor a 0.2 correspondiente el plano bidimensional de CP3 y CP4.

Es importante mencionar que se pueden seguir diversos criterios para seleccionar el valor mínimo absoluto que una carga debe tener para considerarla importante. Uno de los más sencillos es considerar el número total (n) de variables y aplicar la ecuación $1/\sqrt{n}$, lo que en este caso nos daría como resultado $1/\sqrt{71} \approx 0.12$. Existen criterios más estrictos como el *estadístico de Hair* que consideran valores de hasta 0.5 para considerar que una variable tiene significancia práctica. Sin embargo, como se ha mencionado a lo largo del texto, al tratarse de un análisis exploratorio este valor se puede establecer según los objetivos de la investigación.

Interpretación del ACP

Al observar las Figuras 9, 10 y 11 es posible deducir conclusiones químicas sobre las muestras de vino. En particular, en el espacio bidimensional de CP_1 y CP_2 (Figura 9-A), las muestras de Australia (triángulos verdes) tienden a aglomerarse en el cuadrante superior izquierdo. Al inspeccionar la gráfica de cargas en la misma zona (Figura 10), observamos una fuerte influencia de variables como el 9-decenoato de etilo (Ethyl 9-decenoate), el propionato de amilo (Amylpropionate) y el ácido tartárico (TartaricA). Esto nos sugiere que el perfil del vino australiano se caracteriza por una mayor concentración de estos ésteres volátiles y por

la acidez tartárica en comparación con las muestras de los otros países. Por el contrario, variables como el ácido volátil (**VolatileA**) “llevan” las muestras hacia el cuadrante inferior.

Por otra parte, al inspeccionar el espacio bidimensional conformado por el CP_3 y CP_4 (Figura 9-B), se observa un escenario distinto: las muestras analizadas no exhiben agrupamientos ni patrones de discriminación basados en su origen geográfico (todos los puntos se traslapan). Esto nos puede indicar que el 17.6% de la varianza que capturan estos ejes no está vinculada a factores regionales. Al evaluar el gráfico de cargas correspondiente (Figura 11), observamos que las variables con mayor peso en este plano son variables como el glicerol (**Glycerol**), el ácido málico (**MalicACid**), el ácido láctico (**LacticAc**) y subproductos volátiles como el ácido butírico (**Butanoic acid**) y el piruvato de etilo (**Ethyl pyruvate**). Estos compuestos se pueden considerar como marcadores metabólicos del proceso de fermentación y el hecho de que no contribuyan a la diferenciación entre regiones puede atribuirse a la estandarización de las prácticas de vinificación modernas (donde bodegas de distintas latitudes emplean cepas de levaduras comerciales y protocolos analíticos similares). Esto se traduce en que la variabilidad de estos compuestos químicos tiende a ser homogénea a nivel global.

Con todo lo anterior, el *ACP* demuestra su versatilidad no solo para identificar diferencias en el conjunto de datos (CP_1 y CP_2), sino también para aislar y reconocer aquellos rasgos químicos en común que unifican a distintos tipos vino independientemente de su frontera geográfica (CP_3 y CP_4).

ACCELERAR EL FLUJO DE TRABAJO

Todo el código anterior tuvo como objetivo principal ilustrar a detalle las bases del *ACP*. Para análisis más directos y rápidos es posible utilizar funciones dentro *R base* (sin necesidad de instalar ningún paquete) tales como `prcomp()` y `princomp()`, así como paquetes dedicados a este análisis como **PCAtools**, **easyCODA** y **FactoMineR**. (Greenacre et al., 2022) Se recomienda explorar la documentación de estos paquetes y funciones si se requiere de un análisis sin tantas líneas de código. También es importante mencionar que en varios casos el pre-procesamiento y correcto formato de los datos puede requerir de otras herramientas, tal como se muestra (de forma muy breve) en la sección correspondiente a la importación de datos de las muestras de vino.

CONCLUSIONES

La utilización de metodologías analíticas como el *FT-IR* y la *CG-MS* genera conjuntos de datos masivos cuya interpretación mediante estadística univariada puede resultar complicada. El análisis de componentes principales se considera como una metodología exploratoria multivariada cuyo objetivo es reducir la dimensionalidad de los datos originales, transformando múltiples variables correlacionadas en un número reducido de componentes ortogonales que aglomeran la mayor parte de la variación observada. La utilización de *R* para llevar a cabo este tipo de análisis ofrece versatilidad y un gran conjunto de herramientas de libre acceso, lo que permite llevar cabo la importación de datos, el pre-procesamiento y escalado de los mismos, así como los cálculos para obtener las puntuaciones de las muestras y las cargas de las variables junto a las gráficas correspondientes en dos dimensiones. En conjunto, todo lo anterior permite no solo discriminar la identidad o el origen de las muestras o unidades experimentales, sino también identificar de manera directa los marcadores químicos responsables de dichas diferencias.

VERSIONES DE LOS PAQUETES UTILIZADOS

Para garantizar la reproducibilidad de los resultados y gráficas presentados en este tutorial, a continuación se detallan las versiones de los paquetes de *R* utilizados:

Paquete	Versión
ggplot2	4.0.3
ggpubr	0.6.3
dplyr	1.2.1
readr	2.2.0
purrr	1.2.2
stringr	1.6.0
ggrepel	0.9.8

DECLARACIÓN SOBRE LA DISPONIBILIDAD DE LOS DATOS

Los datos de vino utilizados en este artículo son de acceso de libre y pueden descargarse en su formato original del sitio <https://ucphchemometrics.com/2023/06/01/wine-samples-analyzed-by-gc-ms-and-ft-ir-instruments/>. Además, el autor convirtió dichos datos a un formato más accesible (CSV), los cuales pueden ser descargados del repositorio <https://github.com/juanpa-ch/PCA-CIENCIAS-QUIMICAS-DATOS>.

AGRADECIMIENTOS

El autor agradece la beca de posdoctorado (CVU 689991) otorgada por la Secretaría de Ciencia, Humanidades, Tecnología e Innovación (SECIHTI). Lo anterior hizo posible la realización de este artículo de difusión.

DECLARACIÓN DEL AUTOR

El contenido de este artículo es una modificación de lo publicado en el blog personal del autor *R in the Lab*. Actualmente, dicho blog ya no existe y cabe destacar que se llevaron a cabo modificaciones sustanciales en el texto y código para darle mayor sustento y claridad.

Uso de IA asistiva: Durante la preparación de este trabajo, el autor utilizó el modelo Gemini (Google) con el único objetivo de mejorar la redacción, ortografía y eficiencia técnica del código computacional. Tras este proceso, el autor revisó, editó y tomó total responsabilidad del contenido y conclusiones científicas publicadas.

REFERENCIAS

- Bro, R., & Smilde, A. K. (2014). Principal component analysis. *Anal. Methods*, 6(9), 2812–2831. <https://doi.org/10.1039/C3AY41907J>
- Fernández, V., Berradre, M., Sulbarán, B., Ojeda de Rodríguez, G., & Peña, J. (2009). Caracterización química y contenido mineral en vinos comerciales venezolanos. *Revista de La Facultad de Agronomía*, 26(3), 382–397. http://ve.scielo.org/scielo.php?script=sci_abstract&pid=S0378-78182009000300005&lng=es&nrm=iso&tlng=es
- Fernández-Chuairey, D. C. L., Oca, L. R.-M. de, Varela-Nualles, M., & Pino-Roque, J. A. (2022). Analysis of Main Components, an Effective Tool in Agricultural Technical Sciences. *Revista Ciencias Técnicas Agropecuarias*, 31(1), 101–107. <https://revistas.unah.edu.cu/index.php/rcta/article/view/1551>

- Greenacre, M., Groenen, P. J. F., Hastie, T., D’Enza, A. I., Markos, A., & Tuzhilina, E. (2022). Principal component analysis. *Nature Reviews Methods Primers*, 2(1), 100. <https://doi.org/10.1038/s43586-022-00184-w>
- Grijalva-Verdugo, C., Hernández-Martínez, M., Meza-Márquez, O. G., Gallardo-Velázquez, T., & Osorio-Revilla, G. (2018). FT-MIR spectroscopy and multivariate analysis for determination of bioactive compounds and antioxidant capacity in Cabernet Sauvignon wines. *CyTA - Journal of Food*, 16(1), 561–569. <https://doi.org/10.1080/19476337.2018.1428224>
- Pati, S., Tufariello, M., Crupi, P., Coletta, A., Grieco, F., & Losito, I. (2021). Quantification of Volatile Compounds in Wines by HS-SPME-GC/MS: Critical Issues and Use of Multivariate Statistics in Method Optimization. *Processes*, 9(4), 662. <https://doi.org/10.3390/pr9040662>
- R Core Team. (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Svok, T., D., B., & Bro, R. (2008). Multiblock variance partitioning. A new approach for comparing variation in multiple data blocks. In *Chemometrics Research*. <https://ucphchemometrics.com/2023/06/01/wine-samples-analyzed-by-gc-ms-and-ft-ir-instruments/>
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., . . . Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>
- Wold, S., Esbensen, K., & Geladi, P. (1987). Principal Component Analysis. *Chemometrics and Intelligent Laboratory Systems*, 2, 37–52. [https://doi.org/10.1016/0169-7439\(87\)80084-9](https://doi.org/10.1016/0169-7439(87)80084-9)